Field Hours

CSCE 749 Applied Cryptography

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Write the function before the protocol

You start applied cryptography by writing the function you want, not by naming a product. Two parties hold inputs x and y. You want both to learn f(x, y) and nothing else you can avoid. For a toy, x is 4, y is 1, and f is the sum, so the output is 5. If either party also learns the other's input, you computed a different function, the pair, and you should say so.

You name the adversary before you pick a tool. A semi-honest party follows the messages and tries to read more. A malicious party can send a wrong message. A definition that covers only the first does not cover the second. You also name the leak you accept, such as a circuit depth of 100 gates. A protocol with no function and no leak list is not applied yet.

Pin the goal, then the leak

1. Write f of the inputs

2. Name who learns the output

3. List the accepted leak

4. Only then pick a tool

Sum five, or the whole pair

1. Party A holds 4 and party B holds 1. The agreed function is addition. The honest output is 5.

2. A transcript that also shows B the value 4 computed the pair, not the sum. You reject that transcript for this goal.

3. You write the leak you do accept: both parties learn that the output is 5, and they learn that two inputs were used. They do not learn the other input.

Problem 1. A holds 4, B holds 1, and f is addition. The protocol reveals 4 to B and then prints 5. Did you compute f?

Answer. No. f returns only 5. Revealing 4 means you computed something closer to the pair (4, 5). The extra input is not an allowed leak unless you wrote it down as one.

Problem 2. Your notes name a product and never name f or the adversary. What is missing?

Answer. The function, the leak list, and whether the other party is only curious or can send bad messages. A product name does not answer any of those, even if the circuit has 100 gates.

Lecture 2 · 90 minutes

Commit, then open

A commitment lets you bind to a value now and open it later. The toy uses a hash: you pick a 4-bit pad and a 4-bit value, and the commitment is the hash of pad followed by value. Opening means you send both parts, and the other person hashes them again. Binding means you should not find a second value that opens the same commitment. Hiding means the commitment alone should not show the value.

You do not ship the toy. A 4-bit pad has only 16 choices, so the hiding is a sketch. The mechanism is still the one you need: the pad is random, and the open is checked by hashing again. A real pad is long enough that guessing is not the exercise. If you can open the same commitment to 2 values, binding has already failed.

Hide now, open later

Commit

Random 4-bit pad

Hash pad and value

Send only the hash

Open

Send pad and value

Recompute the hash

Reject a mismatch

Sixteen pads, one value

1. You choose a 4-bit pad and a 4-bit value. There are 16 possible pads. You send the hash and keep both parts.

2. Later you open by sending the pad and the value. The checker hashes them and compares to the stored hash.

3. A second value with some pad that hits the same hash would break binding. With a tiny toy hash you do not claim that cannot happen.

Problem 1. The pad is 4 bits. How many pads exist, and why is that not a real hiding parameter?

Answer. There are 16 pads. A search of 16 is small, so this toy only shows commit-then-open. You would not ship a 4-bit pad and call the value hidden.

Problem 2. You open with a pad and a value, and the recomputed hash does not match. What do you do?

Answer. You reject the opening. The check is equality with the commitment. A mismatched recompute is not a successful open, even if the value looks plausible.

Lecture 3 · 90 minutes

Convince without handing over the witness

A zero-knowledge proof is a protocol for a statement and a witness. The statement might be that a commitment opens to an even 4-bit value. The witness is the pad and the value. Completeness says an honest prover convinces the verifier. Soundness says a prover with no witness convinces you only with tiny probability. Zero knowledge says the transcript can be simulated without the witness, so the verifier learns the statement and not the 4 bits.

You keep the three properties apart when you read a design. A proof that sends the witness can be complete and sound, but it is not zero knowledge. In the even-value story the verifier may learn 1 bit, the parity, and not the other bits. If the transcript shows the pad, you leaked the witness. A simulator with no witness is the test that those bits stayed hidden.

Three properties, kept apart

Complete: honest proof works

Sound: no witness, tiny chance

Zero knowledge: simulate it

Parity without the four bits

1. The statement is that the committed 4-bit value is even. That is 1 bit of information you are willing to reveal.

2. The prover has the pad and the value. Sending both would let the verifier check, and would also reveal all 4 bits. That fails zero knowledge.

3. A simulator that can build a matching transcript with no witness is the reason you may claim the extra bits stayed hidden. You still need a soundness argument.

Problem 1. The prover sends the 4-bit value and the pad, and the verifier checks the commitment and the parity. Which property fails?

Answer. Zero knowledge. The check can be complete and sound, but the transcript contains the witness. The verifier was only supposed to learn 1 bit, the parity.

Problem 2. A simulator with no witness produces a transcript the verifier cannot tell from a real one. What does that support, and what does it not replace?

Answer. It supports the zero-knowledge property. It does not replace soundness. A prover with no witness must still have only a tiny chance of convincing you.

Lecture 4 · 90 minutes

A short proof for a long circuit

You compile the statement into a circuit, or an equivalent set of constraints, and a prover who knows the witness produces a short proof. The verifier checks the proof and does not replay every gate. A classroom size makes the split obvious: the circuit has 10000 gates, and the proof stays under 1 kilobyte. The verifier's work stays small

compared with redoing those 10000 gates. That asymmetry is the point of a succinct argument.

You still verify the statement, not a vibe. The proof shows that some witness satisfies the circuit you fixed in a public setup or in the verifier key. If you encoded the wrong circuit, a valid proof checks the wrong thing. Setup, when the scheme needs one, is a trust step you name: 1 toxic value thrown away, or a ceremony you can audit. You do not skip that sentence because the proof is short.

Circuit, proof, then one check

Fix the circuit in public -> Prover uses the witness -> Proof stays under 1 kilobyte -> Verifier skips the 10000 gates

Ten thousand gates, one short proof

1. The statement compiles to 10000 gates. Re-running them is the cost you want the verifier to avoid.

2. The prover holds the witness and emits a proof under 1 kilobyte. The witness is not attached.

3. The verifier checks the proof against the circuit key. A pass means this circuit accepted, not some other circuit you meant to write.

Problem 1. The circuit has 10000 gates and the proof is under 1 kilobyte. What work does the verifier skip, and what do they still bind the proof to?

Answer. They skip replaying the 10000 gates. They still bind the proof to the circuit in the verifier key. A short proof of the wrong circuit is a valid proof of the wrong statement.

Problem 2. The scheme needs a setup value that must be deleted. You keep the value because the proof is small. What did you add?

Answer. A trusted party who can forge, namely anyone who still holds that 1 setup value. Shortness does not remove the setup obligation. You name the ceremony or you do not claim the soundness you think you have.

Lecture 5 · 90 minutes

Outsource the work, keep the check

Verifiable computation lets you send an input, let a server do the work, and check a proof instead of redoing it. Take a function that sums 8 numbers. The server returns the sum and a proof that the sum matches those inputs. You check the proof. If the check fails, you discard the sum, even when the number looks familiar.

You still hold the inputs you care about, or a commitment to them, and you run the verifier. If checking costs more than adding the 8 numbers, you picked a small example: the savings show up on a heavy query, not on a handful of additions. The server learns inputs you sent in the clear. You have not bought privacy yet.

Delegate, prove, then accept

1. Send the input

2. Server returns output and proof

3. Check the proof

4. Drop the output on failure

Eight numbers and a sum

1. The client holds 8 numbers that add to 36. It asks the server to compute the sum.

2. The server returns 36 and a proof tied to those inputs. The client runs the verifier and accepts.

3. A second server returns 35 with a proof that does not check. The client keeps 36 from the first result only if that proof passed, and it drops 35.

Problem 1. The server returns 35 for inputs that sum to 36, and the proof check fails. Which number do you keep?

Answer. Neither from that response. You discard 35 because the check failed. The correct sum is 36 only if some proof for these inputs actually verifies. A familiar number is not a check.

Problem 2. You sent 8 inputs in the clear and verified the sum. What privacy did you get?

Answer. None for those inputs. The server saw them. Verifiable computation checks correctness. Hiding the inputs is a separate goal you have not specified.

Lecture 6 · 90 minutes

Add shares without showing the secret

Additive sharing splits a secret so that each party holds noise and the noises sum to the secret. In the toy, values live modulo 10. You want secret 7. You draw share a = 3 and set share b = 4, because 3 plus 4 is 7. Party A sees only 3. Party B sees only 4. Either share alone is compatible with any secret, as long as the other share adjusts. Both shares together reconstruct 7.

Addition of two secrets is local. One secret is split as 3 and 4, another as 1 and 2. Each party adds its shares: 3 plus 1 is 4, and 4 plus 2 is 6. The new shares sum to 10, which is 0 modulo 10, matching 7 plus 3. Multiplication is not local. You need a pre-shared triple and a short exchange. You do not skip that step.

Split, add locally, reconstruct

Pick share a at random -> Set b so a plus b matches -> Add shares on each side -> Open only when you should

Secret seven, shares three and four

1. You split 7 into a = 3 and b = 4 modulo 10. Each party stores one share.

2. A second secret 3 is split into 1 and 2. Local sums are 3 plus 1 = 4, and 4 plus 2 = 6.

3. You open 4 and 6 only when the function says the output may be revealed. They sum to 10, which is 0 modulo 10, the same as 7 plus 3.

Problem 1. Shares are a = 6 and b = 8, modulo 10. What secret do they reconstruct?

Answer. 4. Six plus 8 is 14, and 14 modulo 10 is 4. You need both shares. One of them is not enough.

Problem 2. Secrets 7 and 3 are shared as (3, 4) and (1, 2) modulo 10. What are the local sums, and

what output do they open to?

Answer. Local sums are 4 and 6. They open to 0 modulo 10, because 4 plus 6 is 10. That matches 7 plus 3, which is also 10, or 0 modulo 10.

Lecture 7 · 90 minutes

Hide a record inside a training step

Private learning names which object stays hidden. If 4 clients each hold a gradient and you only need the sum, you can add the gradients with shares so the server sees the sum and not one client's vector. That is secure computation applied to an aggregate. It is not the same as hiding the final model. The model may still memorize a record. You do not let the word private do both jobs.

Differential privacy is the other mechanism: you add noise so that one record moves the released numbers only inside a budget you set. You pick epsilon before you train. A smaller epsilon is a tighter promise and usually more noise. An epsilon of 1 is a budget you spend when you release a result, not a label on a slide. It limits influence. It does not hide a person who is the only member of a group of 1.

Two different privacy goals

Hidden gradients

Four clients share a sum

Server sees the aggregate

The model can still memorize

Bounded influence

Noise under a budget

Epsilon 1 is one budget

A group of 1 can stick out

Four gradients, one sum

1. Four clients hold gradients. You want the server to learn the sum and not client 2's vector.

2. Each client shares its gradient. Local addition produces shares of the sum. The server opens only the sum.

3. You still have not set epsilon. If you later release a model, you pick epsilon, here 1, and you spend it on that release. The aggregate protocol did not spend it for you.

Problem 1. Four clients sum gradients through shares, and the server opens only the sum. Does client 2's gradient stay hidden from the server?

Answer. Yes, under the sharing assumption: the server sees the sum, not client 2's vector. The final model is a different object and may still memorize a record. This protocol did not set epsilon.

Problem 2. You release a model under epsilon 1 and one person is the only member of a group. What

did epsilon 1 not do?

Answer. It did not make that person blend into a crowd. Epsilon 1 bounds how much one record can move the released numbers. A group of 1 can still stick out. You had to pick the budget before release.

Lecture 8 · 90 minutes

A private computation you can now specify

You can now specify a private computation without treating a chain or a model as magic. You write the function, the leak, and the adversary. You can commit and open by recomputing, and you know a 4-bit pad has only 16 choices. You can ask for a proof that is complete, sound, and simulated. A proof under 1 kilobyte can stand in for a circuit of 10000 gates, and you reject a sum whose proof fails.

You can also split a secret modulo 10, add shares locally, and open only the output. Shares 6 and 8 reconstruct 4, because 14 modulo 10 is 4, and one share fixes nothing. On a learning task you can sum 4 clients' gradients, and you separately spend epsilon 1 if you release a model. Put a proof on a chain only when the contract checks this statement. Say what the verifier learns.

The spec you can now write

Function, leak, adversary

Commit, prove, or share

Check before you accept

Name epsilon if you release

Shares, then a checked release

1. Two parties hold shares 6 and 8 modulo 10. Opened together they reconstruct 4. Opened alone, either share fits every secret.

2. They also want a public checker to accept the result. They produce a short proof for the circuit that says the opened sum is 4, and the checker does not receive the shares.

3. If a model is released afterward, you set epsilon to 1 up front. The proof check does not choose that budget.

Problem 1. Shares modulo 10 are 6 and 8. You commit to each share and open both, and a proof check for the sum succeeds. What is the reconstructed secret, and why was one share not enough?

Answer. The secret is 4, because 6 plus 8 is 14 and 14 modulo 10 is 4. One share is not enough: the missing share can be any of 10 digits, so the secret is not fixed until both open. The proof must be for that sum, not for a different circuit.

Problem 2. The same result is a model release, and you never set epsilon, but the proof was under 1 kilobyte for 10000 gates. What is still unspecified?

Answer. The privacy budget. A short proof shows the circuit accepted. It does not add noise or set epsilon. If you meant epsilon 1, you have to choose it before release and say so.